

# Systems Programming and Computer Architecture

Laurin Seeholzer

September 17, 2026

# Contents

|          |                                 |          |
|----------|---------------------------------|----------|
| <b>1</b> | <b>C Programming</b>            | <b>3</b> |
| 1.1      | History and Toolchain . . . . . | 3        |
| 1.2      | Control flow . . . . .          | 3        |
| 1.3      | Basic types in C . . . . .      | 4        |
| 1.4      | Operators . . . . .             | 5        |
| 1.5      | Arrays . . . . .                | 5        |

# 1 C Programming

## 1.1 History and Toolchain

C is very fast, offers preprocessing and is close to metal. What you won't find in C are objects, classes, traits, features, methods, interfaces, fancy types, exception handling.

**Important 1.1.** Most importantly C does not offer memory management, it uses pointers that offer direct access to memory addresses.

We'll use C99, the most commonly used variant, in this course.

### Code Example 1.2

```
1 #include <stdio.h>
2
3 int main(int argc, char *argv[])
4 {
5     printf("hello, world\n");
6     return 0;
7 }
```

## 1.2 Control flow

### Code Example 1.3

```
1 if (Boolean expression) Statement_when_true
2     else Statement_when_false
3
4 switch (Integer expression) {
5     case Constant_1: Statement; break;
6     ...
7     default: Statement; break;
8 }
9
10 for (Initial; Test; Increment) Statement
11
12 while (Boolean expression) Statement
13
14 do Statement while (Boolean expression)
15
16 goto Label
17
18 printf()
19
20 return (Expression)
```

**Important 1.4. Don't use goto:** It's almost never a good idea, usually argued on performance.

**Remark 1.5.** `goto` can be used to break out of multiple nested loops or to cleanup nested code.

### 1.3 Basic types in C

#### Code Example 1.6

```
1      {static, const} type variable = value;
```

**Remark 1.7.** The prefix **const** before a variable ensures it can not be changed lateron.

**Definition 1.8. Scope:** The scope of a variable in C is the Block it is defined in (lexically typed).

**Remark 1.9.** The prefix **static** before a variable definition ensures the value persists between function calls. Note that static limits the scope of a variable to the file/compilation unit.

**Definition 1.10. Integers & Floats:** Size of these differ from system to system

- **char** is always 1 Byte
- **int & float** are always 4 Bytes
- **long long & double** are always 8 Bytes

**Definition 1.11. Booleans:** Boolean values in C are integers where zero = false, non-zero = true

**Definition 1.12. Void:** Void is a type without a value and can be used for raw memory pointers or functions without returnvalue.

**Definition 1.13. enums:** used to enumerate/sequentially assign values to variables.

#### Code Example 1.14

```
1  eum { ThisIsZero, ThisIsOne, ThisIsTwo } numbers;
```

## 1.4 Operators

### Code Example 1.15

```
1  () []
2  ! ~ ++ -- + - * & (type) sizeof
3  * / %
4  + -
5  << >>
6  < <= > >=
7  == !=
8  &
9  ^
10 |
11 &&
12 ||
13 ?:
14 = += -= *= /= %= &= ^= |= <<= >>=
15 ,
```

**Remark 1.16.** Early termination in C means expressions such as  $A \& B$  never evaluate B if A is already evaluated to false.

**Definition 1.17. Ternary Operator:** ternary operators allow for concise conditional assignment.

### Code Example 1.18

```
1 int a = {test}? ifTrue : ifFalse;
```

**Definition 1.19. Assignment Operator:** In C an assignment to a variable is not just a statement its an expressison that evaluateates to the value that is assigned.

**Definition 1.20. Post- & Pre-Increment/Decement:** As in many languages the difference between  $a++$  and  $++a$  is the value of the expression (before/after incrementing)

**Important 1.21. Casting** of most types to most other types in possible in C. What exactly happens is very complicated and determined by the types involved.

## 1.5 Arrays